

TOPIC 2 : BRUTE FORCE

Question: 1 Write a program to perform the following:

- An empty list
- A list with one element
- A list with all identical elements
- A list with negative numbers

Test Cases:

1. Input: [] → Expected Output: []
2. Input: [1] → Expected Output: [1]
3. Input: [7, 7, 7, 7] → Expected Output: [7, 7, 7, 7]
4. Input: [-5, -1, -3, -2, -4] → Expected Output: [-5, -4, -3, -2, -1]

Aim

To write a Python program to sort a list using the **Bubble Sort (Brute Force)** algorithm and test it with different types of input lists.

Algorithm

1. Start.
2. Read the input list.
3. Find the length of the list.
4. Compare adjacent elements.
5. If the left element is greater than the right element, swap them.
6. Repeat the process until the list is sorted.
7. Print the sorted list.
8. Stop.

Code:

```
1 def bubbleSort(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(n - i - 1):
5             if arr[j] > arr[j + 1]:
6                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
7
8     return arr
9 # Test Case 1
10 print(bubbleSort([]))
11
12 # Test Case 2
13 print(bubbleSort([1]))
14
15 # Test Case 3
16 print(bubbleSort([7, 7, 7, 7]))
17
18 # Test Case 4
19 print(bubbleSort([-5, -1, -3, -2, -4]))
```

input

```
[]
[1]
[7, 7, 7, 7]
[-5, -4, -3, -2, -1]
```

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Question 2: Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order.

Test Cases

1. **Input:** [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
2. **Input:** [10, 8, 6, 4, 2]
Output: [2, 4, 6, 8, 10]
3. **Input:** [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

Aim

To write a Python program to sort an array using the **Selection Sort** algorithm.

Algorithm

1. Start.
2. Read the input array.
3. Assume the first unsorted element is the minimum.
4. Compare it with the remaining unsorted elements.
5. Find the smallest element.
6. Swap it with the first unsorted element.
7. Repeat until the array is completely sorted.
8. Print the sorted array.
9. Stop.

Example: 1

```

1- def selectionSort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         min_index = i
6-
7-         for j in range(i + 1, n):
8-             if arr[j] < arr[min_index]:
9-                 min_index = j
10-
11-         arr[i], arr[min_index] = arr[min_index], arr[i]
12-
13-     return arr
14-
15-
16- arr = [5, 2, 9, 1, 5, 6]
17- print(selectionSort(arr))

```

[1, 2, 5, 5, 6, 9]

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Example : 2

```

1- def selectionSort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         min_index = i
6-
7-         for j in range(i + 1, n):
8-             if arr[j] < arr[min_index]:
9-                 min_index = j
10-
11-         arr[i], arr[min_index] = arr[min_index], arr[i]
12-
13-     return arr
14-
15-
16- arr = [10, 8, 6, 4, 2]
17- print(selectionSort(arr))

```

[2, 4, 6, 8, 10]

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Example : 3

```
1 def selectionSort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         min_index = i
6
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_index]:
9                 min_index = j
10
11         arr[i], arr[min_index] = arr[min_index], arr[i]
12
13     return arr
14
15
16 arr = [1, 2, 3, 4, 5]
17 print(selectionSort(arr))
```



[1, 2, 3, 4, 5]

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Question 3: Write code to modify the Bubble Sort function to stop early if the list becomes sorted before all passes are completed.

Test Cases

1. **Input:** [64, 25, 12, 22, 11]
Output: [11, 12, 22, 25, 64]
2. **Input:** [29, 10, 14, 37, 13]
Output: [10, 13, 14, 29, 37]
3. **Input:** [3, 5, 2, 1, 4]
Output: [1, 2, 3, 4, 5]
4. **Input:** [1, 2, 3, 4, 5] (*Already Sorted List*)
Output: [1, 2, 3, 4, 5]
5. **Input:** [5, 4, 3, 2, 1] (*Reverse Sorted List*)
Output: [1, 2, 3, 4, 5]

Aim

To write a Python program to implement an **optimized Bubble Sort** that stops early if the array is already sorted.

Algorithm

1. Start.
2. Read the input array.
3. Traverse the array using Bubble Sort.
4. Set a flag swapped = False before each pass.
5. Compare adjacent elements and swap if necessary.
6. If any swap occurs, set swapped = True.
7. If no swaps occur in a pass, stop the algorithm because the array is already sorted.
8. Print the sorted array.
9. Stop.

Example: 1

```
1 - def bubbleSort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8             if arr[j] > arr[j + 1]:
9                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
10                swapped = True
11
12        if not swapped:
13            break
14
15    return arr
16
17    arr = [64, 25, 12, 22, 11]
18
19    print(bubbleSort(arr))
```

[11, 12, 22, 25, 64]

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Example: 2

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Question 4: Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

Example 1

Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Example 2

Input: [5, 5, 5, 5, 5]

Output: [5, 5, 5, 5, 5]

Example 3

Input: [2, 3, 1, 3, 2, 1, 1, 3]

Output: [1, 1, 1, 2, 2, 3, 3, 3]

Aim

To write a Python program to sort an array containing duplicate elements using the **Insertion Sort** algorithm while preserving the relative order of duplicate elements (stable sorting).

Algorithm

1. Start.
2. Read the input array.
3. Traverse the array from the second element.
4. Store the current element as the key.
5. Compare the key with previous elements.
6. Shift larger elements one position to the right.
7. Insert the key into its correct position.
8. Repeat until the array is completely sorted.
9. Print the sorted array.
10. Stop.

Example 1

```
1 def insertionSort(arr):
2
3     for i in range(1, len(arr)):
4         key = arr[i]
5         j = i - 1
6
7         while j >= 0 and arr[j] > key:
8             arr[j + 1] = arr[j]
9             j -= 1
10
11        arr[j + 1] = key
12
13    return arr
14
15
16 arr = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
17
18 print(insertionSort(arr))
```

[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Example : 3

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Question 5: Given an array `arr` of positive integers sorted in a strictly increasing order and an integer `k`, return the `k`th positive integer that is missing from this array.

Example 1

Input: `arr = [2,3,4,7,11]`, `k = 5`

Output: 9

Explanation:

The missing positive integers are [1,5,6,8,9,10,12,...].

The **5th** missing positive integer is **9**.

Example 2

Input: `arr = [1,2,3,4]`, `k = 2`

Output: 6

Explanation:

The missing positive integers are [5,6,7,...].

The **2nd** missing positive integer is **6**.

Aim

To write a Python program to find the **kth missing positive integer** from a sorted array of positive integers.

Algorithm

1. Start.
2. Read the array `arr` and integer `k`.
3. Initialize the current positive number as 1.
4. Traverse through the array.
5. Count the missing positive integers until the `k`th missing number is found.
6. If all array elements are processed and `k` is still not reached, continue counting.
7. Print the `k`th missing positive integer.
8. Stop.

Example 1

```
1 def findKthPositive(arr, k):
2     current = 1
3     i = 0
4
5     while k > 0:
6         if i < len(arr) and arr[i] == current:
7             i += 1
8         else:
9             k -= 1
10            if k == 0:
11                return current
12            current += 1
13
14
15 arr = [2, 3, 4, 7, 11]
16 k = 5
17
18 print(findKthPositive(arr, k))
```

9

Example: 2

```
1 def findKthPositive(arr, k):
2     current = 1
3     i = 0
4
5     while k > 0:
6         if i < len(arr) and arr[i] == current:
7             i += 1
8         else:
9             k -= 1
10            if k == 0:
11                return current
12            current += 1
13
14
15 arr = [1, 2, 3, 4]
16 k = 2
17
18 print(findKthPositive(arr, k))
```

6

Time Complexity

$O(n + k)$

Space Complexity

$O(1)$

Question 6: A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer array `nums`, find a peak element and return its index. If the array contains multiple peaks, return the index of any one of them. You must write an algorithm that runs in $O(\log n)$ time.

Example 1

Input: `nums = [1,2,3,1]`

Output: 2

Explanation: 3 is a peak element, so its index is 2.

Example 2

Input: `nums = [1,2,1,3,5,6,4]`

Output: 5

Explanation: The peak element is 6, whose index is 5.

Aim

To write a Python program to find the index of a **peak element** in an array using **Binary Search**.

Algorithm

1. Start.
2. Read the input array `nums`.
3. Initialize `left = 0` and `right = n - 1`.
4. Find the middle element.
5. If the middle element is smaller than the next element, search the right half.
6. Otherwise, search the left half.
7. Continue until `left == right`.
8. Print the index of the peak element.
9. Stop.

Example 1

```
1 def findPeakElement(nums):
2     left = 0
3     right = len(nums) - 1
4
5     while left < right:
6         mid = (left + right) // 2
7
8         if nums[mid] < nums[mid + 1]:
9             left = mid + 1
10        else:
11            right = mid
12
13    return left
14
15
16    nums = [1, 2, 3, 1]
17
18    print(findPeakElement(nums))
```

Time Complexity

$O(\log n)$

Space Complexity

$O(1)$

Example : 2

```
1 def findPeakElement(nums):
2     left = 0
3     right = len(nums) - 1
4
5     while left < right:
6         mid = (left + right) // 2
7
8         if nums[mid] < nums[mid + 1]:
9             left = mid + 1
10        else:
11            right = mid
12
13    return left
14
15
16    nums = [1, 2, 1, 3, 5, 6, 4]
17
18    print(findPeakElement(nums))
```

Time Complexity

$O(\log n)$

Space Complexity

$O(1)$

Question 7: Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at indices **0** and **6**. The first occurrence is at index **0**.

Example 2

Input: haystack = "leetcode", needle = "leeto"

Output: -1

Explanation: "leeto" is not present in "leetcode".

Aim

To write a Python program to find the first occurrence of a substring (needle) in another string (haystack) using the **Brute Force** approach.

Algorithm

1. Start.
2. Read the strings haystack and needle.
3. Traverse the haystack from the first character to the last possible starting position.
4. Compare each character of needle with the corresponding characters in haystack.
5. If all characters match, return the current index.
6. If no match is found after checking all positions, return -1.
7. Print the result.
8. Stop.

Example: 1

```
1 - def strStr(haystack, needle):
2
3     n = len(haystack)
4     m = len(needle)
5
6     for i in range(n - m + 1):
7         if haystack[i:i + m] == needle:
8             return i
9
10    return -1
11
12
13    haystack = "sadbutsad"
14    needle = "sad"
15
16    print(strStr(haystack, needle))
```

Time Complexity

$O(n \times m)$

Space Complexity

$O(1)$

Example 2

```
1 - def strStr(haystack, needle):
2
3     n = len(haystack)
4     m = len(needle)
5
6     for i in range(n - m + 1):
7         if haystack[i:i + m] == needle:
8             return i
9
10    return -1
11
12
13    haystack = "leetcode"
14    needle = "leeto"
15
16    print(strStr(haystack, needle))
```

Time Complexity

$O(n \times m)$

Space Complexity

$O(1)$

Question 8: Given an array of string words, return all strings in words that are a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string.

Example 1

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is a substring of "mass" and "hero" is a substring of "superhero".

Example 2

Input: words = ["leetcode","et","code"]

Output: ["et","code"]

Explanation: "et" and "code" are substrings of "leetcode".

Example 3

Input: words = ["blue","green","bu"]

Output: []

Explanation: No string is a substring of another string.

Aim

To write a Python program to find all strings in an array that are substrings of another string using the Brute Force approach.

Algorithm

1. Start.
2. Read the list of strings.
3. Compare each string with every other string.
4. If one string is found as a substring of another string, add it to the result.
5. Continue until all strings are checked.
6. Print the list of substrings.
7. Stop.

Example 1

```
1 def stringMatching(words):
2     result = []
3
4     for i in range(len(words)):
5         for j in range(len(words)):
6             if i != j and words[i] in words[j]:
7                 result.append(words[i])
8                 break
9
10    return result
11
12 words = ["mass", "as", "hero", "superhero"]
13
14 print(stringMatching(words))
```

['as', 'hero']

Time Complexity

$O(n^2 \times m)$

Space Complexity

$O(k)$

Example: 2

```
1 def stringMatching(words):
2     result = []
3
4     for i in range(len(words)):
5         for j in range(len(words)):
6             if i != j and words[i] in words[j]:
7                 result.append(words[i])
8                 break
9
10    return result
11
12 words = ["leetcode", "et", "code"]
13
14 print(stringMatching(words))
```

['et', 'code']

Time Complexity

$O(n^2 \times m)$

Space Complexity

$O(1)$ (excluding the output list)

Question 9: Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input

A list or array of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output

The two points with the minimum distance between them.

Closest Pair: (1, 2) - (3, 1)

Minimum Distance: 2.23606797749979

Note: The distance shown in your question (1.4142135623730951) is mathematically incorrect for the given points. The correct Euclidean distance between (1,2) and (3,1) is $\sqrt{5} \approx 2.23606797749979$.

Aim

To write a Python program to find the closest pair of points in a set of 2D points using the Brute Force approach.

Algorithm

1. Start.
2. Read the list of 2D points.
3. Initialize the minimum distance as infinity.
4. Compare every pair of points.
5. Calculate the Euclidean distance between each pair.
6. If the current distance is smaller than the minimum distance, update the minimum distance and closest pair.
7. Print the closest pair and the minimum distance.
8. Stop.

Example

```
1 import math
2 def closestPair(points):
3     min_distance = float('inf')
4     pair = None
5     n = len(points)
6     for i in range(n):
7         for j in range(i + 1, n):
8             x1, y1 = points[i]
9             x2, y2 = points[j]
10            distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
11            if distance < min_distance:
12                min_distance = distance
13                pair = (points[i], points[j])
14        return pair, min_distance
15 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
16 pair, distance = closestPair(points)
17 print("Closest Pair:", pair)
18 print("Minimum Distance:", distance)
```

input

```
Closest Pair: ((1, 2), (3, 1))
Minimum Distance: 2.23606797749979
```

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Question 10: Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Write a brute-force algorithm to solve the convex hull problem for the given set of points.

Given Points

- P1 (10, 0)
- P2 (11, 5)
- P3 (5, 3)
- P4 (9, 3.5)
- P5 (15, 3)
- P6 (12.5, 7)
- P7 (6, 6.5)
- P8 (7.5, 4.5)

Expected Output

Convex Hull:

P3, P4, P6, P5, P7, P1

Aim

To write a Python program to find the Convex Hull of a given set of 2D points using the Brute Force approach.

Algorithm

1. Start.
2. Read the list of points.
3. Compare every pair of points to form a line.
4. Check whether all the remaining points lie on the same side of the line.
5. If they do, the line is part of the convex hull.
6. Collect all hull points.
7. Remove duplicate points.
8. Print the convex hull points.
9. Stop.

Example :1

```
1- def orientation(p, q, r):
2-     return (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])
3- def convexHull(points):
4-     n = len(points)
5-     hull = set()
6-     for i in range(n):
7-         for j in range(i + 1, n):
8-             pos = neg = 0
9-             for k in range(n):
10-                 if k == i or k == j:
11-                     continue
12-                 val = orientation(points[i], points[j], points[k])
13-                 if val > 0:
14-                     pos += 1
15-                 elif val < 0:
16-                     neg += 1
17-                 if pos == 0 or neg == 0:
18-                     hull.add(points[i])
19-                     hull.add(points[j])
20-     return list(hull)
21- points = [

```

input

Convex Hull Points:
[(6, 6.5), (10, 0), (15, 3), (5, 3), (12.5, 7)]

Time Complexity

$O(n^3)$

Space Complexity

$O(n)$

Question 11: Write a program that finds the convex hull of a set of 2D points using the brute force approach.

Input

A list of points represented by coordinates (x, y).

Points: [(1,1), (4,6), (8,1), (0,0), (3,3)]

Expected Output

Convex Hull: [(0,0), (1,1), (8,1), (4,6)]

Aim

To write a Python program to find the **Convex Hull** of a set of 2D points using the **Brute Force** approach.

Algorithm

1. Start.
2. Read the list of 2D points.
3. Consider every pair of points as a possible edge.
4. Check whether all the remaining points lie on the same side of the line formed by the pair.
5. If they do, include both points in the convex hull.
6. Remove duplicate hull points.
7. Print the convex hull points.
8. Stop.

Example: 1

```
1 def orientation(p, q, r):
2     return (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])
3 def convexHull(points):
4     n = len(points)
5     hull = set()
6     for i in range(n):
7         for j in range(i + 1, n):
8             positive = 0
9             negative = 0
10            for k in range(n):
11                if k == i or k == j:
12                    continue
13                value = orientation(points[i], points[j], points[k])
14                if value > 0:
15                    positive += 1
16                elif value < 0:
17                    negative += 1
18            if positive == 0 or negative == 0:
19                hull.add(points[i])
20                hull.add(points[j])
21        return list(hull)
22 points = [(1,1), (4,6), (8,1), (0,0), (3,3)]
23 print("Convex Hull:")
24 print(convexHull(points))
```

input

Convex Hull:
[(0, 0), (4, 6), (8, 1)]

Time Complexity

$O(n^3)$

Space Complexity

$O(n)$

Question 12: You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the Traveling Salesman Problem (TSP).

The program should:

1. Define a function distance(city1, city2) to calculate the Euclidean distance.
2. Generate all possible permutations of the cities (excluding the starting city).
3. Calculate the total distance for each route.
4. Return the shortest distance and the corresponding path.

Test Case 1

Input:

Cities = [(1,2), (4,5), (7,1), (3,6)]

Expected Output:

Shortest Distance: 7.0710678118654755

Shortest Path:

[(1,2), (4,5), (7,1), (3,6), (1,2)]

Test Case 2

Input:

Cities = [(2,4), (8,1), (1,7), (6,3), (5,9)]

Expected Output:

Shortest Distance: 14.142135623730951

Shortest Path:

[(2,4), (1,7), (6,3), (5,9), (8,1), (2,4)]

Aim

To write a Python program to solve the **Traveling Salesman Problem (TSP)** using the **Brute Force (Exhaustive Search)** approach.

Algorithm

1. Start.
2. Read the coordinates of all cities.
3. Fix the first city as the starting city.
4. Generate all possible permutations of the remaining cities.
5. Calculate the total distance for each route.

6. Keep track of the minimum distance and corresponding path.
7. Print the shortest path and minimum distance.
8. Stop.

Example : 1

```
1 from itertools import permutations
2 from math import sqrt
3 def distance(city1, city2):
4     return sqrt((city1[0]-city2[0])**2 + (city1[1]-city2[1])**2)
5 def tsp(cities):
6     start = cities[0]
7     min_distance = float('inf')
8     best_path = None
9     for perm in permutations(cities[1:]):
10         path = [start] + list(perm) + [start]
11         total = 0
12         for i in range(len(path)-1):
13             total += distance(path[i], path[i+1])
14         if total < min_distance:
15             min_distance = total
16             best_path = path
17     return min_distance, best_path
18 cities = [(1,2), (4,5), (7,1), (3,6)]
19 dist, path = tsp(cities)
20 print("Shortest Distance:", dist)
21 print("Shortest Path:", path)
```

Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Time Complexity

$O(n \times n!)$

Space Complexity

$O(n)$

Question 13: You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the Assignment Problem.

- Define a function `total_cost(assignment, cost_matrix)` to calculate the total assignment cost.
- Generate all possible assignments using permutations.
- Find the assignment with the minimum total cost.

Test Case 1

Input

Cost Matrix =

[[3, 10, 7],

[8, 5, 12],

[4, 6, 9]]

Expected Output

Optimal Assignment:

Worker 1 → Task 1

Worker 2 → Task 2

Worker 3 → Task 3

Total Cost: 17

Note: The assignment given in your question (Total Cost = 19) does **not** produce the minimum cost. The correct minimum cost for this matrix is **17**.

Test Case 2

Input

Cost Matrix =

[[15, 9, 4],

[8, 7, 18],

[6, 12, 11]]

Expected Output

Optimal Assignment:

Worker 1 → Task 3

Worker 2 → Task 2

Worker 3 → Task 1

Total Cost: 17

Note: The assignment in the question states **24**, but the actual minimum cost is **17**.

Aim

To write a Python program to solve the **Assignment Problem** using the **Brute Force (Exhaustive Search)** approach.

Algorithm

1. Start.
2. Read the cost matrix.
3. Generate all possible task assignments using permutations.
4. Calculate the total cost for each assignment.
5. Compare all assignment costs.
6. Store the assignment having the minimum cost.
7. Print the optimal assignment and minimum cost.
8. Stop.



```
1 from itertools import permutations
2 def total_cost(assignment, cost_matrix):
3     cost = 0
4     for worker in range(len(assignment)):
5         cost += cost_matrix[worker][assignment[worker]]
6     return cost
7 def assignment_problem(cost_matrix):
8     n = len(cost_matrix)
9     min_cost = float('inf')
10    best_assignment = None
11    for assignment in permutations(range(n)):
12        cost = total_cost(assignment, cost_matrix)
13        if cost < min_cost:
14            min_cost = cost
15            best_assignment = assignment
16    return best_assignment, min_cost
17 cost_matrix = [
18     [3, 10, 7],
19     [8, 5, 12],
20     [4, 6, 9]
21 ]
22 best_assignment, min_cost = assignment_problem(cost_matrix)
23 print("Optimal Assignment:")
24 for worker, task in enumerate(best_assignment):
25     print(f"Worker {worker+1} -> Task {task+1}")
26 print(f"Total Cost: {min_cost}")
```

input

Time Complexity

$O(n \times n!)$

Space Complexity

$O(n)$

Question 14: You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem.

The program should:

- Define a function `total_value(items, values)` that calculates the total value of selected items.
- Define a function `is_feasible(items, weights, capacity)` that checks whether the total weight exceeds the knapsack capacity.
- Generate all possible subsets of items using exhaustive search.
- Return the subset with the maximum total value without exceeding the capacity.

Test Case 1

Input

Items = [0,1,2]

Weights = [2,3,1]

Values = [4,5,3]

Capacity = 4

Expected Output

Optimal Selection: [0,2]

Total Value: 7

Test Case 2

Input

Items = [0,1,2,3]

Weights = [1,2,3,4]

Values = [2,4,6,3]

Capacity = 6

Expected Output

Optimal Selection: [0,1,2]

Total Value: 12

Note: The question mentions **10**, but the correct total value is **12** because items **0, 1, and 2** have weights **1+2+3 = 6** (within capacity) and values **2+4+6 = 12**.

Aim

To write a Python program to solve the **0-1 Knapsack Problem** using the **Brute Force (Exhaustive Search)** approach.

Algorithm

1. Start.
2. Read the weights, values, and knapsack capacity.
3. Generate all possible subsets of items.
4. Calculate the total weight and total value of each subset.
5. Ignore subsets whose weight exceeds the capacity.
6. Store the subset having the maximum value.
7. Print the optimal subset and its total value.
8. Stop.

```
1 from itertools import combinations
2 def total_value(items, values):
3     return sum(values[i] for i in items)
4 def is_feasible(items, weights, capacity):
5     return sum(weights[i] for i in items) <= capacity
6 def knapsack(weights, values, capacity):
7     n = len(weights)
8     best_items = []
9     best_value = 0
10    for r in range(n + 1):
11        for subset in combinations(range(n), r):
12            if is_feasible(subset, weights, capacity):
13                value = total_value(subset, values)
14                if value > best_value:
15                    best_value = value
16                    best_items = list(subset)
17    return best_items, best_value
18 weights = [2,3,1]
19 values = [4,5,3]
20 capacity = 4
21 items, value = knapsack(weights, values, capacity)
22 print("Optimal Selection:", items)
23 print("Total Value:", value)
```

input

Optimal Selection: [1, 2]
Total Value: 8

Time Complexity ; $O(2^n \times n)$

Space Complexity : $O(n)$

